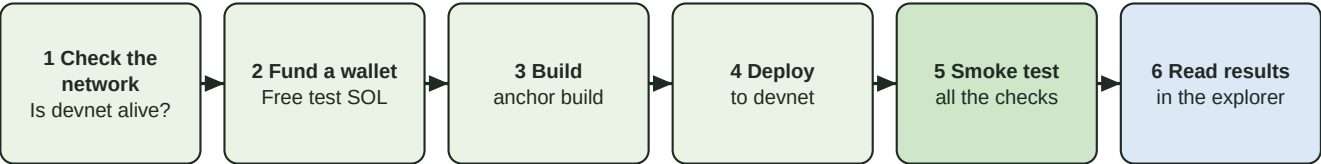


Testing the Solana program on devnet

A terminal guide. Green blocks are commands to type. Output marked **real output** was captured on this machine; output marked **expected** was not run here.

The whole path, in order



Steps 1 to 5 happen in your terminal. Step 6 happens in your browser.

Do the whole path on a local validator first: it is free, instant and offline. Then repeat on devnet, where the network is shared and public.

Figure 1. The path from a clean machine to a verified program.

Three places you can test

	Local validator	Devnet	Mainnet
Cost	Free	Free test SOL	Real money
Speed	Instant, offline	Real network, a few seconds per step	Real network
Shared with others	No, only you	Yes, public	Yes, public
Use it for	Every code change	Proving it works on a real cluster	Not yet. After audit and pilot.

Check your tools first

```
$ solana --version
solana-cli 3.1.13 (src:437252fc; feat:534737035, client:Agave)
$ anchor --version
anchor-cli 0.30.1
```

Real output. Other versions usually work; if the build fails, compare against these.

Step 1: Is devnet alive? (the network checker)

These read-only commands cost nothing and need no wallet. Run them any time something looks wrong: if they fail, the problem is the network or your connection, not our program.

```
$ solana -u devnet cluster-version
4.4.0-beta.0
$ solana -u devnet epoch-info
Block height: 495049569
Slot: 507805687
Epoch: 1175
Epoch Completed Percent: 47.613%
Epoch Completed Time: 13h 33m 37s/1day 4h 35m 5s (15h 1m 28s remaining)
$ solana -u devnet block-height
495049569
$ curl -s https://api.devnet.solana.com -X POST -H 'Content-Type: application/json' \
  -d '{"jsonrpc":"2.0","id":1,"method":"getHealth"}'
{"jsonrpc":"2.0","result":"ok","id":1}
```

Real output. A healthy network answers within a second, **block height keeps rising** when you run it twice a few seconds apart, and getHealth says ok.

Optional: `solana -u devnet ping -c 3` sends tiny real transactions and measures confirmation time. It needs a funded wallet.

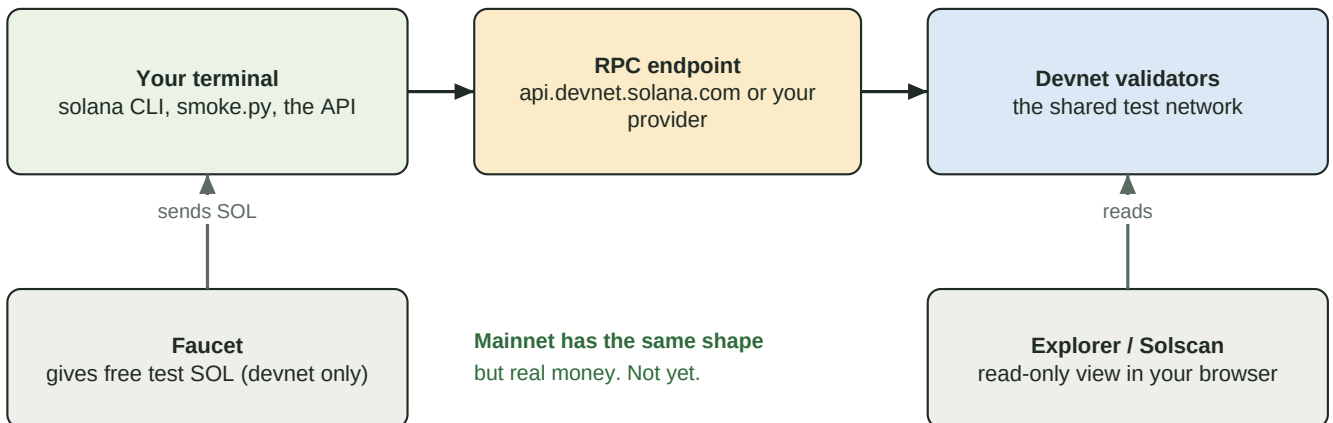


Figure 2. What talks to what. The RPC endpoint is the only thing you may swap later (setting SOLANA_RPC_URL).

Checkers in your browser

Tool	Link	Use it for
Solana Explorer	explorer.solana.com/?cluster=devnet	Paste a transaction signature, program id or account address. Shows instructions, logs, and errors.
Solscan	solscan.io/?cluster=devnet	Same idea with a different interface. Handy as a second opinion.
Solana status page	status.solana.com	Official status. Covers Mainnet Beta only ; it does not list devnet, so use the commands above for devnet.
Devnet faucet	faucet.solana.com	Free devnet SOL. Limit is 2 requests per 8 hours without signing in; a GitHub login raises it.

Step 2: A wallet with test SOL

Add `-u devnet` to each command instead of changing your global setting. Your CLI is currently pointed at a devnet provider, so a global change would only matter if you also use other clusters.

```
$ solana address
<your wallet address>
$ solana -u devnet balance
5 SOL
# below 2 SOL? ask the faucet (2 tries per 8 hours without a GitHub login):
$ solana -u devnet airdrop 2
Requesting airdrop of 2 SOL ... 7 SOL (expected)
```

The balance line is real output from your wallet. A deploy needs about 1.4 SOL of rent plus a few cents of fees, so 5 SOL is plenty.

Step 3: Test locally first

Open **two terminals**. Terminal A runs a private test network with our program already loaded. Terminal B runs the checks.

```
# Terminal A (leave it running; stop any other validator or Surfpool first)
$ cd ~/Dev/singapore
$ ID=9ZZ5bejD4NdPr7zf9covKHDig6tcqh628Fdtrm4mT9MU
$ SO=blockchain/solana/target/deploy/marketplace_settlement.so
$ solana-test-validator --reset --ledger /tmp/sv-ledger --bpf-program $ID $SO

# Terminal B (paste one line at a time, with no blank lines between them)
$ cd ~/Dev/singapore
$ solana-keygen new --no-bip39-passphrase --force -o /tmp/auth.json
$ solana -u localhost airdrop 5 $(solana-keygen pubkey /tmp/auth.json)
$ solana -u localhost program show 9ZZ5bejD4NdPr7zf9covKHDig6tcqh628Fdtrm4mT9MU
$ SOLANA_AUTHORITY_KEYPAIR=/tmp/auth.json .venv/bin/python blockchain/solana/scripts/smoke.py
```

The program show line must print a Program Id before you continue; if it says the program does not exist, the validator in terminal A was not started with `--bpf-program`. Stop the validator with `Ctrl+C` when you finish. The throwaway key never touches your real wallet; do not reuse its seed phrase for anything real.

What a passing run looks like

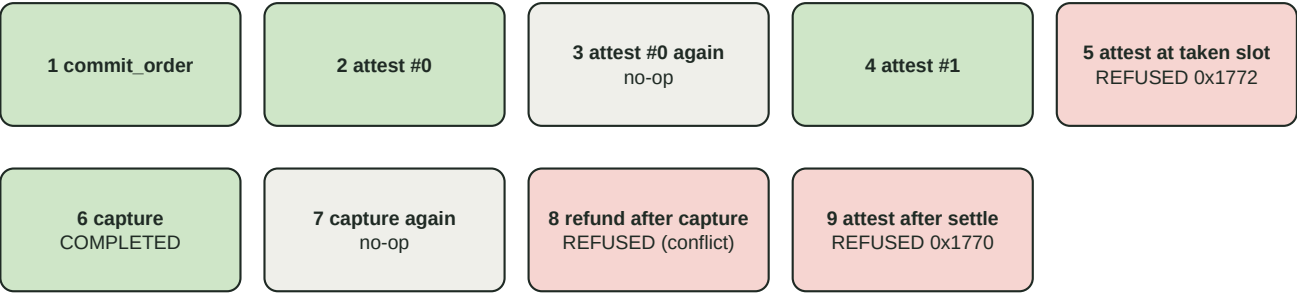
This is real output from the run done while writing this guide, against the rebuilt program. Links are shortened here; in your terminal each one opens the transaction in the explorer.

```
cluster http://127.0.0.1:8899
program 9ZZ5bejD4NdPr7zf9covKHDig6tcqh628Fdtrm4mT9MU
authority EDQ8vRwX...kVS7eSK
order smoke_1791221002
order account CkpRiw21WoZHUwCVci1mjPCGJy5Giy1twCmk4erkkYeB

PASS order not on chain yet: UNKNOWN
PASS commit_order: https://explorer.solana.com/tx/2nBvEA2c...?cluster=custom&customUrl=...
PASS status after commit: COMMITTED
PASS commit again is a no-op: existing
PASS attestation 0: https://explorer.solana.com/tx/Qi2TfGv5...?cluster=custom&customUrl=...
PASS attestation 0 retried is a no-op: existing
PASS attestation 1: https://explorer.solana.com/tx/5LC2auvi...?cluster=custom&customUrl=...
PASS attestation at a taken position, sent raw (refused with 0x1772): True
PASS capture: https://explorer.solana.com/tx/5h7mH5GjwvaCGZYBQ...?cluster=custom&customUrl=...
PASS status after capture: COMPLETED
PASS capture again is a no-op: existing
PASS refund after capture (refused with cannot become REFUNDED): True
PASS attestation after settlement, sent raw (refused with 0x1770): True

all checks passed
```

What the smoke test proves: green passes, grey does nothing on purpose, red is refused on purpose



Steps 5, 8 and 9 are the cases that were untested before Phase 1.
Retries that already landed are now harmless, and conflicting outcomes are refused.

Figure 3. The checks, grouped. Red boxes are supposed to be refused: if one is accepted, the script stops and says so.

Step 4: Build the program

```
$ cd blockchain/solana
$ anchor build --no-idl
Finished `release` profile [optimized] target(s) in 0.33s (real output)
$ solana-keygen pubkey target/deploy/marketplace_settlement-keypair.json
9ZZ5bejD4NdPr7zf9covKHDig6tcqh628Fdtrm4mT9MU
```

The key printed must equal the program id written in the code (`declare_id!`). The keypair file is git-ignored; never commit it, because whoever holds it can claim the program address.

Step 5: Deploy to devnet

```
$ solana program deploy target/deploy/marketplace_settlement.so \
$ --program-id target/deploy/marketplace_settlement-keypair.json -u devnet
Program Id: 9ZZ5bejD4NdPr7zf9covKHDig6tcqh628Fdtrm4mT9MU (expected)
$ solana program show 9ZZ5bejD4NdPr7zf9covKHDig6tcqh628Fdtrm4mT9MU -u devnet
Program Id: 9ZZ5...mT9MU
Owner: BPFLoaderUpgradeab1e1111111111111111111111111111
Authority: <your wallet address> Data Length: 202920 (0x318a8) bytes
```

The show output is real on the local validator; on devnet the Authority is your wallet, which is what lets you upgrade it. Running the same deploy command again upgrades the program in place.

Good to know before you deploy

- **It is public.** Anyone can read a devnet program and call it. That is fine for testing, since only the authority key can change a record.
- **Rent is locked, not spent.** Roughly 1.4 SOL stays locked in the program account. `solana program close` can recover it, but a closed program id can never be reused. Check `solana program close --help` before using it.
- **Version skew is normal.** Devnet reported 4.4.0-beta.0 while your CLI is 3.1.13; that does not affect this program.

Step 6: Run the smoke test on devnet

```
$ cd ~/Dev/singapore
$ SOLANA_RPC_URL=https://api.devnet.solana.com \
$ .venv/bin/python blockchain/solana/scripts/smoke.py
(the same PASS lines as before; the links end in ?cluster=devnet) (expected)
```

It uses your default wallet (~/.config/solana/id.json) as the authority because the script's default is that path. Each commit, attest and capture costs a fraction of a cent.

Step 7: Read the results in the explorer

1. Click or paste any link from the output. The transaction page shows **Success**, the instruction name and the program logs.
2. Paste the program id into the explorer's search box (with the devnet network selected) to see the program and its recent transactions.
3. The run prints an **order account** line near the top. That address holds the order's record; look it up:

```
$ solana account <order account address> -u devnet
Owner: 9ZZ5bejD4NdPr7zf9covKHDig6tcqh628Fdtrm4mT9MU
Length: 118 (0x76) bytes
0050: 01 02 00 00 00 bf fe 45 1a 63 b1 48 7d 45 42 94 .....E.c.H}EB.
      ^^ status = 01 (COMPLETED)    ^^ ^^ ^^ ^^ count = 02 00 00 00 (two evidence hashes)
```

Real output from a local run, trimmed. Byte 80 (row 0050, first value) is the status; bytes 81 to 84 are the evidence count, stored smallest byte first.

What one order record looks like on chain (118 bytes). Amber = what the checker reads.

discriminator bytes 0-7	authority 8-39	order_hash 40-71	amount (cents) 72-79	status 80	attest. count 81-84	evidence chain 85-116	bump 117
----------------------------	-------------------	---------------------	----------------------------	--------------	------------------------	-----------------------------	-------------

status: 0 = COMMITTED, 1 = COMPLETED, 2 = REFUNDED. count: how many evidence hashes were recorded.
evidence chain: each new value is sha256(previous value + new hash).

Figure 4. Bytes 80 and 81 to 84 are the status and the evidence count: the two values the adapter checks before it resends anything.

Watch the program work live

```
# in a second terminal, then run the smoke test again
$ solana logs 9ZZ5bejD4NdPr7zf9covKHDig6tcqh628Fdtrm4mT9MU -u devnet
Program log: Instruction: CommitOrder (expected)
```

When something goes wrong

What you see	What it means	What to do
Attempt to debit an account but found no record of a prior credit	The authority wallet has no SOL on this cluster.	solana -u devnet airdrop 2 <address>, or use the web faucet.
airdrop request failed / 429	Devnet faucet rate limit.	Wait, use faucet.solana.com, or test on the local validator.
custom program error: 0x1770	6000 AlreadySettled: the order is finished.	Expected when testing; in production it means a duplicate settle.
custom program error: 0x1771	6001 BadOutcome: outcome was not 1 or 2.	Only the adapter sends outcomes; this signals a bug.
custom program error: 0x1772	6002 UnexpectedCount: that evidence slot is taken or out of order.	Expected when replaying; the adapter now skips these.
Allocate: account already in use	commit_order was sent twice for one order.	The adapter checks first; this appears only with raw sends.
TimeoutError: not confirmed	The transaction may or may not have landed.	Safe to retry now: the adapter reads the chain first.
ProgramAccountNotFound / program does not exist	The cluster you are talking to does not have our program. On localhost, the validator was started without --bpf-program, or an older validator or Surfpool is still using port 8899.	Stop everything on 8899, restart with the Terminal A commands, check solana -u localhost program show. The smoke script now stops with this message.
authority is not the key you expected	The environment variable and the command were split by a blank line or a missing backslash, so the default wallet was used.	Put SOLANA_AUTHORITY_KEYPAIR=... on the same line as the python command (the guide now does).
Error: Building IDL failed	The nightly Rust on this machine breaks an IDL helper.	Build with anchor build --no-idl. The adapter does not use the IDL.
insufficient funds for rent (deploy)	Deploying needs about 1.4 SOL of rent.	Fund the deployer wallet first.

The three custom program errors are the program's own: 6000 = 0x1770, 6001 = 0x1771, 6002 = 0x1772 (6000 is 0x1770 in hexadecimal).

Optional: test through the API

Start the API with the Solana provider switched on, then run an order through accept, pickup, delivery and complete in the demo app. Every step leaves a row you can read at GET /admin/payments (admin token from /demo/seed?role=admin): **CONFIRMED** with a signature, or **FAILED** with the reason, which POST /admin/payments/retry replays safely. This route was not run in this session; the smoke test above covers the program itself.

```
$ docker compose up -d # local database
$ PAYMENT_PROVIDER=solana SOLANA_RPC_URL=https://api.devnet.solana.com \
$ SOLANA_PROGRAM_ID=9ZZ5bejD4NdPr7zf9covKHDig6tcqh628Fdtrm4mT9MU \
$ SOLANA_AUTHORITY_KEYPAIR=$HOME/.config/solana/id.json \
$ .venv/bin/uvicorn app.main:app --app-dir api --reload
```

Next after this guide: deploy to devnet, run the smoke test there, then move the sending into a worker so it leaves the web request (the rest of Phase 1).